Medical RAG Patient Education Assistant

Project Report

Ashvanth Rathinavel

MS Applied Data Science | University of Southern California

April 2026

Table of Contents

1. Executive Summary

The Medical RAG Patient Education Assistant is an end-to-end Retrieval-Augmented Generation (RAG) system designed to answer patient questions about chronic diseases using authoritative U.S. public health guidelines from the CDC, NHLBI, and NIA. The system ingests, indexes, and retrieves from 106 guideline documents (267 semantic chunks), enforces strict citation grounding to prevent hallucinations, and serves answers through both a Streamlit web interface and a command-line tool.

A comprehensive evaluation benchmark of 30 curated Q&A pairs demonstrates:

Metric	Score
Answer Rate	100.0% (30 / 30)
Retrieval Recall	93.3% (28 / 30)
Citation Rate	100.0% (30 / 30)
Keyword Accuracy	96.7% (29 / 30)
Avg Latency	22.35 seconds

2. Motivation and Problem Statement

Large Language Models (LLMs) are prone to hallucination — generating plausible but factually incorrect medical information. For patient-facing education tools, this is particularly dangerous. Standard LLMs also have knowledge cutoffs and lack grounding in authoritative, up-to-date clinical guidelines.

This project addresses these concerns through four design principles:

- Restricting the knowledge base to verified U.S. government health guidelines (CDC, NHLBI, NIA)
- Enforcing that every generated statement must be explicitly cited to a source chunk
- Refusing to answer when retrieved evidence falls below a confidence threshold
- Using a local LLM backend (Ollama) to avoid data-privacy concerns

Target disease areas: diabetes, hypertension, asthma, COPD, heart disease, stroke, kidney disease, obesity, high cholesterol, tobacco use, alcohol use, arthritis, and Alzheimer's / dementia.

3. System Architecture

3.1 High-Level Overview

The system is divided into two phases: an offline ingestion / indexing pipeline and an online query pipeline.

Offline Pipeline

```
CDC / NHLBI / NIA URLs
  -> Web Scraping  (trafilatura + BeautifulSoup4)
  -> Text Cleaning (rule-based boilerplate removal)
  -> Semantic Chunking (sentence-aware, token-based overlap)
  -> Vector Embedding (BAAI/bge-small-en-v1.5, 384-dim)
  -> Qdrant Index  (cosine similarity, Docker-based)
```

Online Pipeline

```
User Question
  -> Embedding (same BGE-small model)
  -> Cosine Search (Qdrant, top-k=5)
  -> Confidence Gate  (drop if max score < 0.25)
  -> URL Deduplication (keep highest-scoring chunk per URL)
  -> Prompt Construction (numbered [1][2]... context block)
  -> LLM Generation  (Ollama llama3.1, temperature=0)
  -> Citation Enforcement (discard if no [N] citations)
  -> Return answer + source metadata
```

3.2 Technology Stack

Component	Technology
Web Scraping	trafilatura (primary), BeautifulSoup4 (fallback)
Text Cleaning	Custom rule-based cleaner
Chunking	Custom sentence-aware chunker
Embedding Model	BAAI/bge-small-en-v1.5 (384-dim)
Vector Database	Qdrant (Docker, cosine distance)
LLM Backend	Ollama llama3.1 (localhost:11434)
Web UI	Streamlit
Language	Python 3.11+
Containerisation	Docker Compose

4. Data Ingestion Pipeline

4.1 Source URLs

The system scrapes 99 URLs across three authoritative sources:

- **CDC (Centers for Disease Control and Prevention): Primary source for diabetes, heart disease, stroke, COPD, kidney disease, obesity, cholesterol, tobacco, alcohol, and arthritis.**
- **NHLBI (National Heart, Lung, and Blood Institute): Hypertension, heart disease, asthma, COPD, obesity, and cholesterol.**
- **NIA (National Institute on Aging): Alzheimer's disease and dementia.**

Source detection is performed automatically by matching domain patterns in the URL.

4.2 Web Scraping (scrape_cdc.py)

Each URL is fetched with a standard Mozilla user-agent header to avoid bot-detection blocks. Text extraction follows a two-tier approach:

- Primary — trafilatura: A library specifically designed for web content extraction. Handles complex page layouts including tables, preserves paragraph structure, and strips navigation boilerplate automatically.
- Fallback — BeautifulSoup4: Custom HTML parsing that removes script, style, noscript, header, footer, and nav tags before extracting body text.

Document IDs are assigned as sha256(url)[:16] — a 16-character hex prefix of the SHA-256 hash of the source URL. This guarantees stability across re-scrapes (same URL always produces the same ID) and is collision-resistant. Raw HTML is saved to data/raw_html/{doc_id}.html; structured JSON (fields: id, url, title, source, retrieved_at, text) is saved to data/docs/{doc_id}.json. Error stubs are created for failed URLs, enabling targeted retries.

4.3 Text Cleaning (clean_cdc.py)

The cleaning stage applies four transformations in order:

1. Line ending normalisation: converts all CRLF / CR to LF.
2. Navigation junk removal: drops lines under 30 characters that do not end with sentence-terminating punctuation (., !, ?).
3. Boilerplate removal: removes lines containing common CDC footer strings ("about cdc", "contact us", "800-232-4636").
4. Whitespace normalisation: collapses 3+ consecutive blank lines to 2.

Output is saved to data/docs/cleaned/{doc_id}.json with an added char_len field.

4.4 Corpus Statistics

Metric	Value
Documents Ingested	106
Total Chunks Produced	267
Avg Chunk Size (tokens)	294.06
Min Chunk Size (tokens)	82
Max Chunk Size (tokens)	418
Avg Chunk Size (chars)	1,817
Min Chunk Size (chars)	500
Max Chunk Size (chars)	2,781

5. Chunking Strategy

5.1 Design Rationale

Naive fixed-size chunking (e.g. every N characters) can split sentences mid-way, destroying semantic coherence. This project implements a sentence-aware hierarchical chunking strategy that respects natural language boundaries, controlled by the ChunkConfig dataclass.

5.2 ChunkConfig Parameters

Parameter	Value	Description
target_tokens	350	Ideal chunk size
overlap_tokens	60	Context carry-over between consecutive chunks
min_tokens	80	Merge chunks below this threshold into previous
max_tokens	450	Hard upper limit per chunk

5.3 Algorithm (chunking.py)

The chunking pipeline proceeds in three stages:

Stage 1 — Paragraph Splitting (_split_into_paragraphs)

The document text is split into logical paragraphs by detecting blank lines and ALL-CAPS headings. This preserves the document's natural semantic structure and prevents medical headings (e.g. "PREVENTION", "SYMPTOMS") from being buried mid-chunk.

Stage 2 — Sentence Splitting (_split_into_sentences)

Each paragraph is split into individual sentences using a lookbehind regex that matches sentence-ending punctuation followed by whitespace:

```
(?<=[.!?])\s+
```

This avoids splitting on abbreviations (e.g. "Dr. Smith") and handles multiple sentence terminators.

Stage 3 — Greedy Packing with Overlap (_pack_sentences)

Sentences are greedily packed into chunks targeting the configured token size:

- A running token count accumulates sentences until the target (350 tokens) is reached.
- The last 60 tokens of each completed chunk are carried over (prepended) to the next chunk as context overlap.
- Super-long individual sentences exceeding max_tokens are word-split as a fallback.
- Tail chunks under min_tokens (80) are merged into the previous chunk if the combined size stays within max_tokens.

Token counting uses whitespace-based word splitting — an intentional simplification that avoids a hard dependency on an external tokenizer library. The approach is fast, stable, and accurate enough for chunk sizing purposes.

Each output chunk record includes: doc_id, chunk_id, title, url, source, text, char_len, token_est, start_char, end_char.

6. Embedding and Indexing

6.1 Embedding Model

The BAAI/bge-small-en-v1.5 model — from the BGE (BAAI General Embedding) family — is used for all embeddings, both at index time and at query time.

Property	Value
Vector Dimensionality	384
Normalisation	L2-normalised (for cosine similarity)
Language	English
Parameters	~33 M (fast inference, CPU-only)
Library	sentence-transformers

Using the same model for documents and queries at inference time ensures that embedding-space distance is semantically meaningful without any domain shift.

6.2 Qdrant Vector Database

Qdrant is deployed as a Docker container via docker-compose.yml, with data persisted to ./qdrant_data/ via a mounted volume. Key configuration:

Setting	Value
Collection name	medical_guidelines
Distance metric	Cosine similarity
Vector dimension	384
HTTP port	6333
gRPC port	6334
Storage	./qdrant_data/ (persistent volume)

Each indexed point carries the full chunk payload: chunk_id, doc_id, title, url, source, and text. This self-contained design means retrieval results carry all metadata required to build citations and source links without a separate database lookup.

Embeddings are generated in batches of 32 and bulk-upserted to Qdrant. The collection is recreated (not appended to) on each indexing run to guarantee consistency with the current chunk set.

7. Query Pipeline

7.1 Retrieval (retrieve.py)

At query time, the user's question is encoded with the same BAAI/bge-small-en-v1.5 model to produce a 384-dimensional query vector. Qdrant performs an approximate nearest-neighbor search using cosine distance and returns the top-k results (default k=5) with their similarity scores and full payloads.

7.2 Confidence Gate (pipeline.py)

A hard confidence threshold of 0.25 is applied to the maximum similarity score among the retrieved chunks. If max(scores) < 0.25 — or if no results are returned — the pipeline short-circuits and returns the fallback response without ever calling the LLM:

```
"I don't know based on the CDC content I have."
```

This prevents out-of-scope questions (e.g. questions about drug dosages, specific treatments, or topics not covered in the indexed guidelines) from producing hallucinated answers.

7.3 URL Deduplication (pipeline.py)

Before building the prompt, retrieved chunks are deduplicated by source URL. Only the highest-scoring chunk per URL is retained. This prevents the LLM from receiving multiple near-identical excerpts from the same CDC page — which wastes context-window space and causes repetitive [N][N] citation pairs in the response.

7.4 Prompt Construction (prompting.py)

The system prompt sets strict behavioural constraints on the LLM:

- Answer only from the provided CDC excerpts.
- Cite every factual sentence using [N] format (e.g. [1], [2]).
- Return the "I don't know" fallback if the excerpts do not address the question.
- Do not use outside knowledge, add disclaimers, or provide medical advice commentary.

The user prompt presents the question followed by a numbered context block:

```
[1]
{chunk_text_1}

[2]
{chunk_text_2}
...
```

7.5 Generation (generate.py)

The LLM call is made to a local Ollama instance at http://localhost:11434/api/generate.

Parameter	Value
Model	llama3.1 (Meta open-weight)
Temperature	0.0 (fully deterministic)
Stream	False (full response returned at once)
Endpoint	http://localhost:11434/api/generate

The openai and google-generativeai packages are installed but not wired in. generate.py is the single swap point to change the LLM backend to any hosted model without touching any other component.

7.6 Citation Enforcement (pipeline.py)

After generation, a regex scans the answer for [N] citation patterns. The enforcement logic handles three cases:

5. No citations found: answer is discarded; fallback string is returned.
6. Citations found alongside fallback boilerplate: the fallback text is stripped; the cited portion is returned.
7. Citations found, no fallback text: answer is returned as-is.

The cited source indices are used to look up and return the corresponding chunk metadata (title, URL, excerpt) as structured source objects to the UI layer.

8. Interfaces

8.1 Streamlit Web Application (app.py)

A centered Streamlit UI provides the primary user-facing interface:

- Text input field for questions with Enter-key submission support (Streamlit form).
- Loading spinner during inference (st.spinner).
- Generated answer rendered as markdown (st.markdown).
- Expandable source cards below the answer, each showing: source title, clickable URL, and a text excerpt from the matched chunk.

The UI is launched with:

```
streamlit run app.py
```

8.2 Command-Line Interface (scripts/answer.py)

A lightweight CLI wrapper allows quick testing without the browser:

```
python -m scripts.answer "What is the A1C target for adults with diabetes?" --top_k
3
```

Prints the answer and a list of source URLs to stdout. Useful for batch testing and scripted evaluation.

8.3 Direct Retrieval Tester (scripts/test_qdrant_retrieval.py)

A debugging utility that bypasses the LLM entirely and prints the top-k retrieved chunks with scores, titles, URLs, and text previews:

```
python -m scripts.test_qdrant_retrieval "diabetes prevention"
```

This is useful for validating embedding quality and chunk relevance independently of generation.

9. Evaluation

9.1 Benchmark Design (scripts/evaluate.py)

A curated evaluation set of 30 Q&A pairs was developed across 11 medical topic areas. Each entry in data/eval/qa_pairs.json specifies:

- question: The natural-language user query.
- expected_url: The specific CDC / NHLBI page expected to appear in retrieved sources.
- expected_keyword: A key clinical term or value expected to appear in the generated answer.

Topic	Questions
Diabetes	5
High Blood Pressure	3
COPD	3
Obesity	3
Cholesterol	3
Heart Disease	2
Stroke	2
Kidney Disease	2
Alzheimer's	2
Tobacco	2
Alcohol	1

9.2 Metrics

Metric	Definition	Unit
Answer Rate	% of questions receiving a non-fallback answer	%
Retrieval Recall	% where expected_url appears in used sources	%
Citation Rate	% of answers containing at least one [N] citation	%
Keyword Accuracy	% of answers containing expected_keyword (case-insensitive)	%
Avg Latency	Mean end-to-end response time per question	seconds

9.3 Results

Metric	Score
Answer Rate	100.0% (30 / 30)
Retrieval Recall	93.3% (28 / 30)
Citation Rate	100.0% (30 / 30)
Keyword Accuracy	96.7% (29 / 30)
Avg Latency	22.35 s

9.4 Retrieval Misses Analysis

Two questions (6.7%) missed the expected source URL:

- hbp_003 (high blood pressure prevention): The system retrieved a "living-with" page and a kidney-disease prevention page instead of the expected hypertension prevention page. The answer content was clinically accurate, but sourced from adjacent CDC pages.

- diabetes_003 (diabetes statistics): The system retrieved a type-2 diabetes prevention page instead of the expected facts-and-statistics page. The answer was factually correct but not attributed to the expected URL.

Both misses reflect the strictness of URL-level evaluation rather than semantic failure — the retrieved content was topically relevant and the answers were accurate. This suggests the evaluation metric slightly under-reports true answer quality.

9.5 Evaluation Script Behaviour

The script outputs a per-question results table to stdout and optionally writes a JSON results file via the --output flag:

```
python -m scripts.evaluate --output data/eval/results.json
```

The script exits with a non-zero status code if retrieval_recall falls below 50%, enabling integration into CI/CD pipelines as a quality gate.

10. Limitations and Future Work

10.1 Current Limitations

Limitation	Impact
Whitespace-based token counting	Chunk sizes may deviate slightly from targets; accurate enough for current use case
Fixed URL list (99 URLs)	Knowledge base does not update when CDC publishes new guidelines
No query rewriting	Multi-turn or follow-up questions treated as independent queries
No hybrid (BM25 + dense) search	Exact medical terms (drug names, ICD codes) may not retrieve optimally
22-second avg latency	Driven by local Ollama LLM; not suitable for high-throughput use
English-only	Guidelines and embeddings are English; no multilingual support

10.2 Potential Improvements

8. Real tokeniser integration: Replace whitespace token counting with tiktoken or HuggingFace tokenizers for precise chunk sizing that matches the target LLM's vocabulary.
9. Hybrid retrieval: Add BM25 sparse retrieval alongside dense embeddings, merged via Reciprocal Rank Fusion (RRF), to improve recall on exact clinical terms such as drug names and condition codes.
10. Query rewriting: Add a pre-generation step that rewrites multi-part or conversational queries into single retrieval-optimised statements using a lightweight LLM call.
11. Stronger embedding model: Upgrade to bge-large-en-v1.5 or text-embedding-3-small for improved semantic similarity, at the cost of higher compute and latency.
12. LLM upgrade path: Swap Ollama llama3.1 for OpenAI GPT-4o or Gemini 1.5 Pro via the single swap point in generate.py for lower latency and stronger clinical reasoning.
13. Automated re-indexing: Add a scheduled scraping job to detect and ingest updated CDC guidelines automatically, keeping the knowledge base current.
14. RAGAs evaluation: Integrate the ragas framework for LLM-judge-based metrics — faithfulness, answer relevancy, context precision / recall — beyond the keyword-match approach.
15. Cross-encoder reranking: Add a bge-reranker-base reranking step after initial Qdrant retrieval to improve chunk ordering before prompt construction.

11. Project Structure

```
medical_rag/
  app.py                            # Streamlit web UI
  docker-compose.yml                # Qdrant container definition
  requirements.txt                  # Python dependencies
  CLAUDE.md                         # Project documentation
  src/
    ingest/
      cdc_urls.py                   # 99 source URLs
      scrape_cdc.py                # HTML fetching & extraction
      clean_cdc.py                # Text cleaning
      sanity_check.py             # Manual QA tool
    rag/
      pipeline.py                # Orchestration + citation logic
      retrieve.py                # Embedding search
      generate.py                # LLM call (Ollama)
      prompting.py               # System + user prompts
      chunking.py                # Sentence-aware chunking
    utils/
      io.py                      # File I/O helpers
  scripts/
    build_chunks.py              # Generate chunks JSONL
    build_embeddings_qdrant.py   # Index vectors into Qdrant
    answer.py                    # CLI interface
    evaluate.py                  # Benchmark pipeline
    test_qdrant_retrieval.py     # Direct retrieval tester
  data/
    raw_html/                    # Downloaded HTML pages
    docs/
      *.json                     # Scraped documents
      cleaned/                   # Cleaned documents
      chunks/
        medical_chunks.jsonl     # All chunks
        chunk_stats.json         # Corpus statistics
    eval/
      qa_pairs.json              # 30 benchmark questions
      results.json               # Evaluation results
```

12. Dependencies

Package	Role
requests	HTTP fetching for web scraping
beautifulsoup4	HTML parsing (fallback scraper)
lxml	BeautifulSoup backend parser
trafilatura	Primary web content extraction
tqdm	Progress bars for batch operations
sentence-transformers	BGE-small embedding model
qdrant-client	Qdrant vector database Python client
openai	OpenAI API client (installed, not currently wired in)
google-generativeai	Gemini API client (installed, not currently wired in)
streamlit	Web application framework

13. Conclusion

The Medical RAG Patient Education Assistant demonstrates that a tightly-scoped, citation-enforced RAG system can achieve high reliability for domain-specific question answering. By combining authoritative source restriction, dual hallucination safeguards (confidence gate and citation enforcement), and a systematic evaluation framework, the system achieves strong performance: 93.3% retrieval recall and 100% citation adherence on a 30-question benchmark.

The modular architecture — with clearly defined swap points for the LLM backend (generate.py), the embedding model (retrieve.py), and the vector store (retrieve.py + build_embeddings_qdrant.py) — makes the system straightforward to upgrade as better open-weight models become available. The evaluation harness provides a reproducible baseline against which future improvements can be measured.

The primary near-term improvement opportunities are hybrid (BM25 + dense) retrieval for better keyword recall, a real tokeniser for more precise chunking, and cross-encoder reranking to improve context quality before generation.